CS3213: Foundations of Software Engineering

In-class Lecture and Exercises

Bioblitz

- 414 observations
- 159 species
- 5 observers
- 98 identifiers

Collared Scops-Owl

Oriental Whipsnake

Sunda Colugo

Mantis

https://www.inaturalist.org/projects/nus-cs3213-2025

Agenda

- Mid-term feedback
- Mid-term exam solutions and common mistakes
- Test suite quality
- Quality assurance beyond testing

Mid-term Feedback

- Overall, seems going well
  - Learning requirements engineering, ADD, C4
  - Tutorials, pre-recorded lectures
  - Quizzes, PollEv in-class sessions
  - New project direction

Mid-term Feedback

- Common challenges
  - Unfamiliarity with tech stack
  - Challenging to gather requirements, ambiguities with requirements
  - In-class lectures feel a bit slow (some want less, some want more interaction)
    - Lecture recording
  - Tutorials a bit repetitive
  - Discipline with watching videos and doing the quiz
  - Lecture slides/recordings come with little text

Mid-term Feedback

- Some notes
  - Three hours class (reduced lecture slot to two hours due to the blended-learning format)
  - Cannot remove the project component
  - Tradeoff: very clear requirements vs. realism
- Planned changes
  - Announcing assignment releases
  - Lecture notes for next semester (for exam preparation)

Agile Tutorial

Ganesh

Faculty at NUS Singapore, Founder STeAdS, Public Speaker

Singapore
National University of Singapore
Twitter
LinkedIn
Github
Google Scholar
ORCID

Ganesh Neelakanta Iyer Ph.D.

Dr. Ganesh Neelakanta Iyer is a Senior Lecturer in the Department of Computer Science at the National University of Singapore. A dedicated educator and researcher, he bridges the gap between high-level industrial practice and academic excellence, specializing in Agile Software Engineering, Cloud Computing, and Applied Machine Learning.

Education & Academic Background: Dr. Iyer holds a distinguished academic record, having earned his Bachelor's degree in Computer Science and Engineering from Mahatma Gandhi University, Kerala, where he secured the University First Rank. He subsequently moved to Singapore to complete both his Master's (2008) and PhD (2012) at the National University of Singapore.

Prior to his current role at NUS, he held significant academic positions as an Associate Professor at Amrita Vishwa Vidyapeetham and as a Visiting Faculty member at IIIT-Hyderabad.

Industry Expertise: With over a decade of experience in the software industry, Dr. Iyer has worked with global leaders such as Salesforce, NXP Semiconductor, Sasken Communication Technologies, and Progress Software. His multifaceted career has spanned various critical roles, including - QA Architect, Technical Support Manager, Engineering Development Lead, Lead DevOps Engineer and Technology Evangelist.

As a staunch advocate for Agile methodologies, he has managed complex, large-scale business products in roles such as Scrum Master and Product Owner. He now brings this practical expertise into the classroom to prepare the next generation of software engineers.

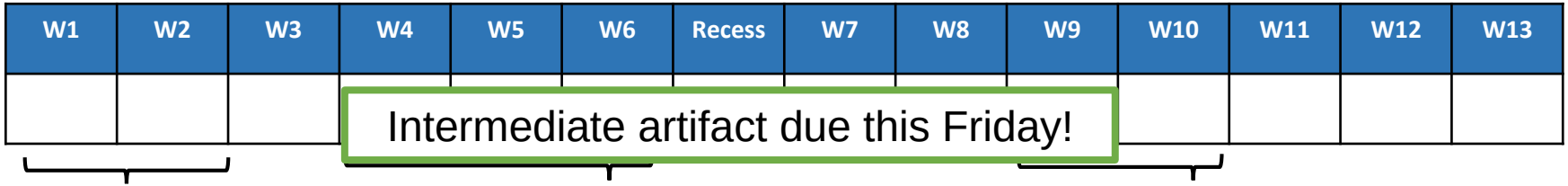
https://ganeshniyer.github.io/

(Planned) Project Assignments

Assignment 2: Requirements Specification
Mon, 26 Jan to Mo, 10 Feb

Assignment 4: Intermediate Artifact
Mon, 2 Mar to Fri, 13 Mar

Assignment 6: Final report and presentation
Mon, 30 Mar to Fri, 17 Apr



Assignment 1: Requirements Elicitation Preparation
Mon, 12 Jan to Fri, 23 Jan

Assignment 3: Design Document
Mon, 2 Feb to Fri, 20 Feb

Assignment 5: Testing
Mon, 16 Mar to Fri, 27 Mar

Week 9: Q/A with CHANG Sau Sheong

W1	W8	W9	W10	W11	W12	W13
L1	L6	L7	L8	L9	L10	

Shaping a digital nation: GovTech's Chang Sau Sheong on leading Singapore's tech evolution

Written by KrASIA Writers
Published on 5 Sep 2024 6 mins read

Share

Photo of Chang Sau Sheong, CTO and deputy chief executive (products) of GovTech.

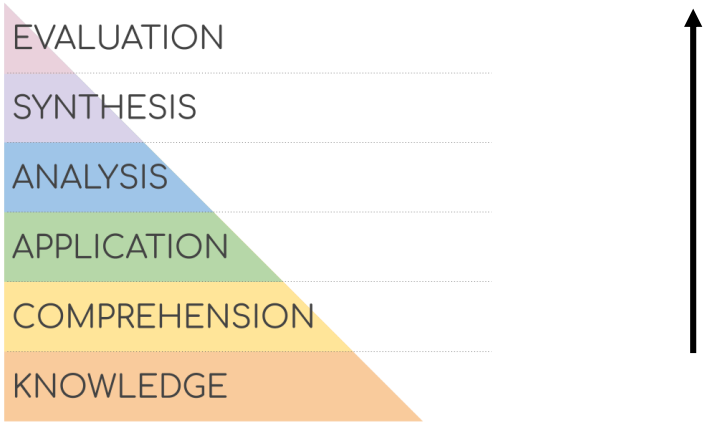
GovTech's CTO discusses Singapore's digital trajectory, highlighting innovation, resilience, and the importance of events like the STACK Developer Conference.

Moved session to W11

https://kr-asia.com/shaping-a-digital-nation-govtechs-chang-sau-sheong-on-leading-singapores-tech-evolution

This Year's Mid-term Exam

- More open-ended questions, few(er) multiple-choice questions
- Focus on the higher layers of Bloom's taxonomy
  - Fewer comprehension/knowledge questions



https://upload.wikimedia.org/wikipedia/commons/7/72/BloomsTaxonomy.png

What is Software Engineering? 1.

(10 points) *The Straits Times* asks you for an interview on why software engineers will still be needed despite increasing AI-based automation in software development. In no more than three sentences, present a brief argument that explicitly references both **essential and accidental complexities** as described in Brooks' "No Silver Bullet" essay.

There are some things that can be automated in software engineering, which we refer to as the accidental complexities, and AI tools primarily address these complexities, for example, by providing a higher-level abstraction for building systems. However, there are some so-called essential complexities that will remain challenging. These include, for example, understanding the business problem and determining the customers' requirements.

What is Software Engineering? 1.

- Common mistakes
  - AI hallucinations or AI mistakes as an example of accidental complexities
  - Explain what essential and accidental complexities are, but do not explain why software engineers are still needed

What is Software Engineering? 2.

(10 points) Assume you are part of the newly established *National Space Agency of Singapore* (NSAS). Your team collaborates with *NASA* (US) and *ESA* (Europe) to develop safety-critical control software for a space shuttle. Give **two reasons**, related to the characteristics of this project, why a plan-driven software engineering approach may be more suitable than an agile approach.

- Safety-critical applications require careful planning.
- Plan-driven approaches are commonly adopted for large-scale projects that span multiple companies.
- Agile works best face-to-face (e.g., daily Scrum meetings).
- Extensive documentation is helpful for such long-running projects.
- Requirements are unlikely to change.
- No pressure to market, and incremental deployment would not be applicable.

What is Software Engineering? 2.

- Common mistakes
  - Describe general agile vs plan-driven differences without connecting them to project
  - Mentioning a general concept, but do not explain why it matters
    - “The system is safety-critical so plan-driven is better.”
  - Incorrect assumptions about agile (e.g., no documentation)

Requirements Engineering. 1.

(12 points) Consider the following two scenarios:
A. Developing a new plastic waste tracking app for NUS students.
B. Integrating an AI chatbot into an existing taxi app (e.g., Grab).

Choose **two requirements elicitation approaches** that would be **applicable mostly to B., but not to A**. For each elicitation approach, specify how you would use it in B., and explain why it is unsuitable for A. Be specific in your reasoning.

- **Document analysis**: As the system already exists, a requirements specification and design document could be studied. Such documents would likely not exist for A, since the system will be newly developed.
- **Workshops***:* As many different stakeholders exist, workshops would be useful to align them and resolve conflicts. With fewer stakeholders, this would likely be an overkill for A.

Requirements Engineering. 1.

- Common mistakes
  - Misunderstanding the concept of a focus group
    - "*A does not have an existing user base, so we cannot conduct a focus group*"
    - *"focus group is in*feasible *for students"*
    - *"focus groups* is not suitable for NUS students because the student population is very large and not all students are interested in plastic waste management.*"*
  - Misunderstanding interview
    - *"impossible to interview every NUS student"*
    - "interviews may seem unnecessary as the target audience is NUS students only,"
  - Document analysis, but mentioning similar applications rather than the mentioning that the documentation and design document might already exist and could be analyzed

Requirements Engineering. 2.

(12 points) Your colleagues ask you to peer-review questions in an interview guide intended for a non-technical stakeholder. For each of the three questions below: (1) Identify the **main problem** with the question (e.g., ambiguous question, double-barrelled question, ...) and (2) **propose an improved phrasing**—making reasonably assumptions about missing information—that would be more appropriate for a non-technical stakeholder.

Question 1: Can you tell me what functionality the system should provide?

- Problem: Too broad
- Improved phrasing: Can you tell me more about how the reporting component should work? What are your biggest pain-points currently?

Requirements Engineering. 2.

(12 points) Your colleagues ask you to peer-review questions in an interview guide intended for a non-technical stakeholder. For each of the three questions below: (1) Identify the **main problem** with the question (e.g., ambiguous question, double-barrelled question, ...) and (2) **propose an improved phrasing**—making reasonably assumptions about missing information—that would be more appropriate for a non-technical stakeholder.

Question 2: I assume you would like to support the application on mobile phones?

- Problem: This is a leading question. The interviewee will more likely say yes.
- Improved phrasing: On what kinds of systems and platforms do you expect the application to run?

Requirements Engineering. 2.

(12 points) Your colleagues ask you to peer-review questions in an interview guide intended for a non-technical stakeholder. For each of the three questions below: (1) Identify the **main problem** with the question (e.g., ambiguous question, double-barrelled question, ...) and (2) **propose an improved phrasing**—making reasonably assumptions about missing information—that would be more appropriate for a non-technical stakeholder.

Question 3: How should the system behave if calling the bank's payment API results in an exception?

- Problem: Technical jargon that the customer might not understand
- Improved phrasing: How should the system behave if the banking transaction fails?

Requirements Engineering. 2.

- Common mistakes
  - Q1: Most identified that the question is too broad, but phrasing was not always improved
  - Q2: Ideally be phrased generally, in devices and platforms

Requirements Engineering. 3.

(8 points) Consider the following requirement: "*The web shop must be highly available.*" Explain in one sentence what the problem with this requirement is, and provide an improved version.

- Issue: this non-functional requirement is not testable
- Improved version: The web shop must be available 99.99% of the time per year.

Software Modeling and Architecture 1.

(4 points) Assume that you want to use a behavioral UML diagram to model **multiple parallel flows**. Which diagram type would be the best choice?

- ○ Sequence diagram
- ○ State machine diagram
- ◉ Activity diagram
- ○ Class diagram

Software Modeling and Architecture 2.

(8 points) Assume you are developing a complex system with many quality attributes. In at most three sentences, explain **how applying Attribute-Driven Design (ADD) might differ** when using an agile approach versus a plan-driven approach.

An agile approach might only iterate one or few times to address the most important quality attributes and mitigate the main risks. In contrast, a plan-driven approach would iterate until all quality attributes have been addressed.

Software Modeling and Architecture 2.

- Common mistakes
  - Not stating that in agile, ADD typically focuses on the most important current quality attributes / architectural drivers and is revisited iteratively.
  - Not stating that in plan-driven, ADD is applied with a more complete upfront view of the relevant quality attributes, so the architecture is planned more comprehensively earlier.
  - Framing the answer around artifacts (user stories, use cases, SRS, tests) rather than around quality attributes / architectural drivers.
  - Talking only about iteration frequency or change tolerance, without connecting that to which quality attributes are addressed and when.

Software Modeling and Architecture 3.

(12 points) You are helping your dentist create an online appointment system. Would it make more sense to implement it as a **layered monolith or using a microservice architecture**? **Justify your choice** by naming at least three factors such as system size, expected number of users, performance, complexity, and/or maintainability.

I would choose a layered monolith. The number of users is small, and a monolith would thus suffice to cope with the performance demands. The complexity of managing multiple executing microservice would cause an additional burden in this scenario. Maintaining the system would also be more difficult, as often, microservices are written in different technologies and frameworks.

Software Modeling and Architecture 3.

- Common mistakes
  - Microservices: overestimating the number of users/scaling needs
    - Small number of deducted points if understood that microservices are typically used for large-scale applications
  - General misunderstanding of monoliths and microservices

Agile Software Engineering 1.

(12 points) Explain in at most five sentences how Scrum, Extreme Programming (XP), and DevOps **could be combined** in practice, and how **their practices can complement** each other. Mention at least **one named concept or practice** from each of the three approaches for illustration.

Extreme programming gives concrete best practices for programming, like pair programming or test-driven development that can be adopted. Scrum is complementary, as it is a general framework for managing projects (e.g., developing the software in “Increments”) and does not provide any best practices for programming. Finally, DevOps is complementary to the other two frameworks by specifying how to deploy Increments in a safe and efficient manner.

Agile Software Engineering 1.

- Common mistakes
  - Listing examples from Scrum, XP, and DevOps without an analysis of the complementary relationship between them
  - Treating a specific step/mindset of Scrum as its overall purpose (e.g., promoting feedback) without mentioning that it is a project management framework

Agile Software Engineering 2.

(12 points) Consider the following principle from the Agile Manifesto: "*Working software is the primary measure of progress.*" Explain **how Scrum reflects this principle**, referencing **two named concepts or practices** from Scrum, in at most three sentences.

Scrum delivers an Increment in every Sprint. This Increment should maximize value and should be potentially deployable.

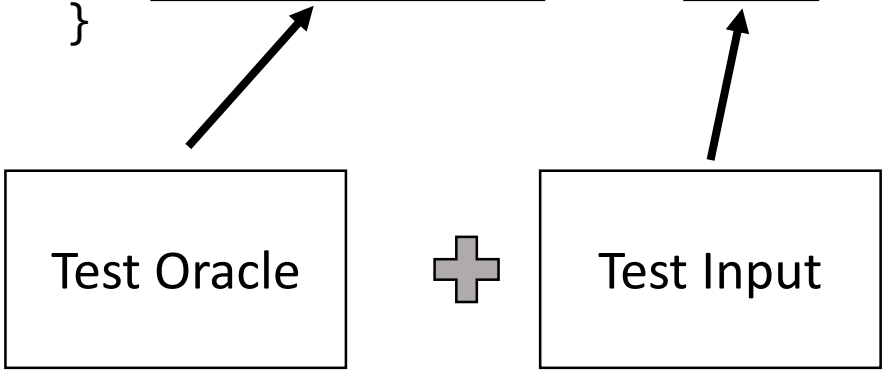
Agile Software Engineering 2.

- Common mistakes
  - Not explicitly mentioning two concepts or practices as required

Recap Software Testing

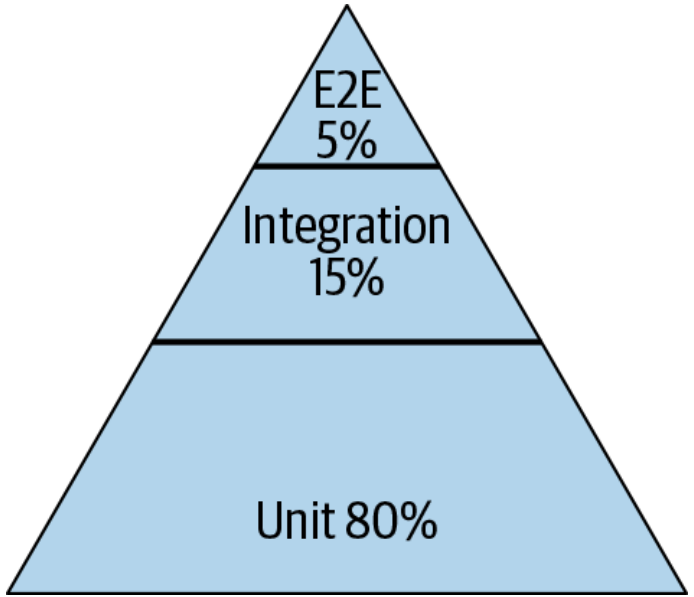
Test Case

```
@Test
public void testAdd() {
    assertEquals(5, add(2, 3));
}
```



Google’s Test Pyramid

- Write tests with different granularity
- Fewer high-level tests
- Many small and fast unit tests

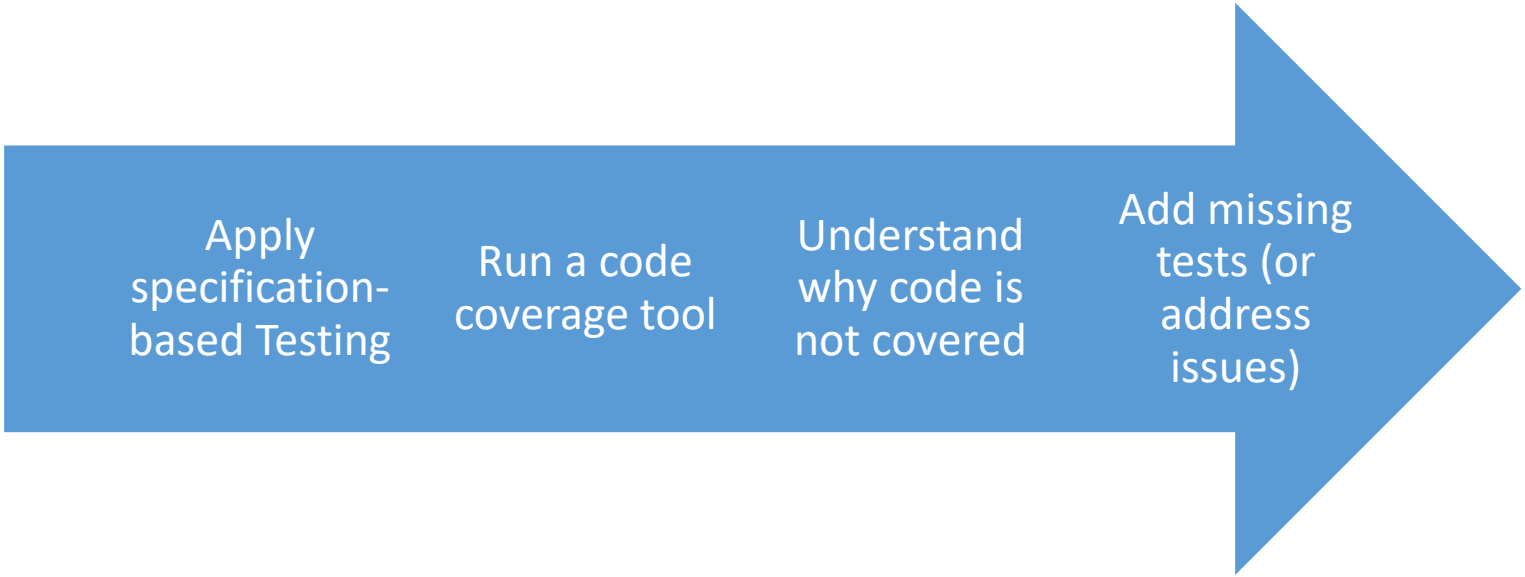


https://abseil.io/resources/swe-book/html/ch11.html

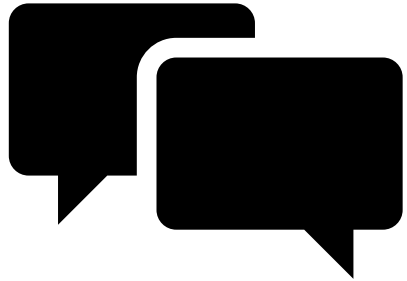
Testing Workflow

1. Understand the requirements
2. Explore the program
3. Identify the partitions
4. Analyze the boundaries
5. Devise test cases
6. Automate test cases
7. Augment (creativity and experience)

Applying Structural Testing



Course Exercise



How can we evaluate the quality of a test suite?

https://pollev.com/manuelrigger

Quality of a Test Suite

- Tests for each requirement
- Test granularity rate
- Code coverage
- Execution time → Extreme Programming
- Flakiness Rate
- **Mutation score**

Mutation Testing

Mutation Score

- Idea: mutate code in the program, assuming that a test case "kills" the mutant
- Percentage of mutants killed → quality of the test suite

$$\text{Mutation score} = \frac{\#\ killed\ mutants}{\#\ mutants} \text{ x } 100$$

Mutation Testing Steps

1. Select statement in the source code
2. Apply a mutation to it (i.e., create a mutant)
3. Execute the test suite
4. Proceed depending on the outcome
    1. Test case fails: mutant is killed
    2. Test case succeeds: mutant survives
5. Undo the change and continue with 1. until a threshold
6. Return the mutation score

$$\text{Mutation score} = \frac{\#\ killed\ mutants}{\#\ mutants} \times 100$$

Mutation Score

```
def bubble_sort(arr):
  length = len(arr)
  for i in range(length):
    for j in range(0, length - i - 1):
      if arr[j] > arr[j+1]:
        arr[j], arr[j+1] = arr[j+1], arr[j]
```

i

<

Step 1: create mutants

Mutation Score

```
def bubble_sort(arr):
  length = len(arr)
  for i in range(length):
    for j in range(0, length - i - 1):
      if arr[j] > arr[j+1]:
        arr[j], arr[j+1] = arr[j+1], arr[i]
```

```
run_tests()
```

Step 2: run test suite for every mutant: mutants is **killed** if test cases fails

Mutation Score

```
def bubble_sort(arr):
  length = len(arr)
  for i in range(length):
    for j in range(0, length - i - 1):
      if arr[j] > arr[j+1]:
        arr[j], arr[j+1] = arr[j+1], arr[i]
```

Mutation score = $\frac{\#\ killed\ mutants}{\#\ mutants}$ x 100

Step 3: Compute a **mutation score** based on the percentage of killed mutants

Mutation Testing

- Advantage: effectiveness
- Disadvantages
  - Computationally expensive
  - Equivalent mutants

Practical Mutation Testing at Scale

A view from Google

Goran Petrović, Marko Ivanković, Gordon Fraser, René Just

Abstract—Mutation analysis assesses a test suite's adequacy by measuring its ability to detect small artificial faults, systematically seeded into the tested program. Mutation analysis is considered one of the strongest test-adequacy criteria. Mutation testing builds on top of mutation analysis and is a testing technique that uses mutants as test goals to create or improve a test suite. Mutation testing has long been considered intractable because the sheer number of mutants that can be created represents an insurmountable problem—both in terms of human and computational effort. This has hindered the adoption of mutation testing as an industry standard. For example, Google has a codebase of two billion lines of code and more than 150,000,000 tests are executed on a daily basis. The traditional approach to mutation testing does not scale to such an environment; even existing solutions to speed up mutation analysis are insufficient to make it computationally feasible at such a scale.

To address these challenges, this paper presents a scalable approach to mutation testing based on the following main ideas: (1) mutation testing is done incrementally, mutating only *changed code* during code review, rather than the entire code base; (2) mutants are filtered, removing mutants that are likely to be irrelevant to developers, and limiting the number of mutants per line and per code review process; (3) mutants are selected based on the historical performance of mutation operators, further eliminating irrelevant mutants and improving mutant quality. This paper empirically validates the proposed approach by analyzing its effectiveness in a code-review-based setting, used by more than 24,000 developers on more than 1,000 projects. The results show that the proposed approach produces orders of magnitude fewer mutants and that context-based mutant filtering and selection improve mutant quality and actionability. Overall, the proposed approach represents a mutation testing framework that seamlessly integrates into the software development workflow and is applicable to industrial settings of any size.

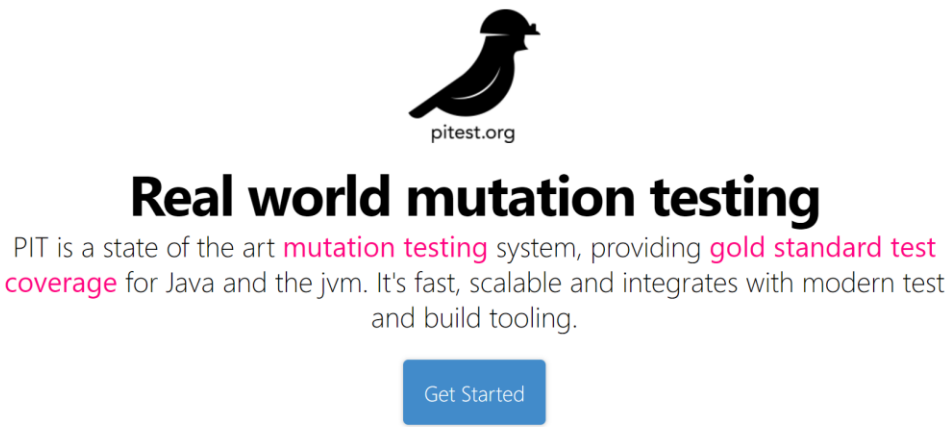
Index Terms—mutation testing, code coverage, test efficacy

https://ieeexplore.ieee.org/document/9524503

Mutation Testing

- Goal
  - Evaluate the quality of existing tests
  - Derive new tests
- White-box technique

Pitest



Eclipse integration: https://github.com/pitest/pitclipse

Pitest: Example for Mutation Operation

Original conditional	Mutated conditional
<	<=
<=	<
>	>=
>=	>

For example

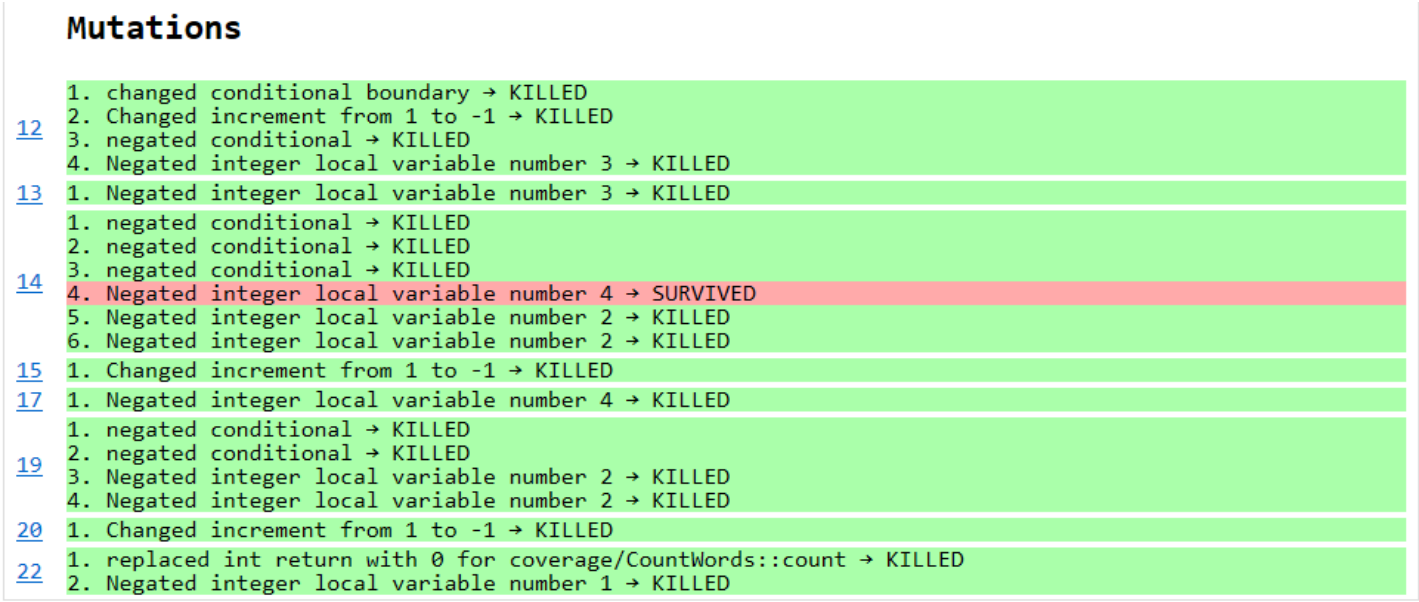
```
if (a < b) {
  // do something
}
```

will be mutated to

```
if (a <= b) {
  // do something
}
```

https://pitest.org/quickstart/mutators/

Pitest



Mutations

12	1. changed conditional boundary → KILLED 2. Changed increment from 1 to -1 → KILLED 3. negated conditional → KILLED 4. Negated integer local variable number 3 → KILLED
13	1. Negated integer local variable number 3 → KILLED
14	1. negated conditional → KILLED 2. negated conditional → KILLED 3. negated conditional → KILLED 4. Negated integer local variable number 4 → SURVIVED 5. Negated integer local variable number 2 → KILLED 6. Negated integer local variable number 2 → KILLED
15	1. Changed increment from 1 to -1 → KILLED
17	1. Negated integer local variable number 4 → KILLED
19	1. negated conditional → KILLED 2. negated conditional → KILLED 3. Negated integer local variable number 2 → KILLED 4. Negated integer local variable number 2 → KILLED
20	1. Changed increment from 1 to -1 → KILLED
22	1. replaced int return with 0 for coverage/CountWords::count → KILLED 2. Negated integer local variable number 1 → KILLED

```
>> Line Coverage: 10/28 (36%)
>> Generated 43 mutations Killed 19 (44%)
>> Mutations with no coverage 23. Test strength 95%
>> Ran 22 tests (0.51 tests per mutation)
```

Pitest

```
8
9           public static int count(String str) {
10                  int nrWords = 0;
11                  char lastChar = 0;
12 4                for (int i = 0; i < str.length(); i++) {
13 1                        char currentChar = str.charAt(i);
14 6                        if (!Character.isLetter(currentChar) && (lastChar == 'r' || lastChar == 's')) {
15 1                                nrWords++;
16                          }
17 1                        lastChar = currentChar;
18                  }
19 4                if (lastChar == 'r' || lastChar == 's') {
20 1                        nrWords++;
21                  }
22 2                return nrWords;
23          }
24
25  }
```

Pitest

- Mutating the program as below does not fail any test case
- Indicates a gap that we can fix

```
if (!Character.isLetter(-currentChar) && (lastChar == 'r' || lastChar == 's')) {
    nrWords++;
}
```

MC/DC Coverage

MC/DC Coverage

- Each decision takes every possible outcome
  - Decision: Boolean expression consisting of conditions and zero or more operators
    - In contrast to branches, also covers assignments to Boolean variables
- Each condition in a decision takes every possible outcome
- Each condition in a decision is shown to independently affect the outcome of the decision

MC/DC Example: A XOR B XOR C

Test case number	*A*	B	C	*decision*
1	F	F	F	F
2	F	F	T	T
3	F	T	F	T
4	F	T	T	F
5	T	F	F	T
6	T	F	T	F
7	T	T	F	F
8	T	T	T	T

How many tests do we need?

MC/DC Example: A XOR B XOR C

Test case number	*A*	B	C	*decision*
1	F	F	F	F
2	F	F	T	T
3	F	T	F	T
4	F	T	T	F
5	T	F	F	T
6	T	F	T	F
7	T	T	F	F
8	T	T	T	T

- Each condition in a decision takes every possible outcome
- Each condition in a decision is shown to independently affect the outcome of the decision

MC/DC Example: A XOR B XOR C

Test case number	*A*	B	C	*decision*
1	F	F	F	F
2	F	F	T	T
3	F	T	F	T
4	F	T	T	F
5	T	F	F	T
6	T	F	T	F
7	T	T	F	F
8	T	T	T	T

Potential tests:
A: {T1, T5}

- Each condition in a decision takes every possible outcome
- Each condition in a decision is shown to independently affect the outcome of the decision

MC/DC Example: A XOR B XOR C

Test case number	*A*	B	C	*decision*
1	F	F	F	F
2	F	F	T	T
3	F	T	F	T
4	F	T	T	F
5	T	F	F	T
6	T	F	T	F
7	T	T	F	F
8	T	T	T	T

Potential tests:
A: {T1, T5}, {T2, T6}

- Each condition in a decision takes every possible outcome
- Each condition in a decision is shown to independently affect the outcome of the decision

MC/DC Example: A XOR B XOR C

Test case number	*A*	B	C	*decision*
1	F	F	F	F
2	F	F	T	T
3	F	T	F	T
4	F	T	T	F
5	T	F	F	T
6	T	F	T	F
7	T	T	F	F
8	T	T	T	T

Potential tests:
A: {T1, T5}, {T2, T6}, {T3, T7},

- Each condition in a decision takes every possible outcome
- Each condition in a decision is shown to independently affect the outcome of the decision

MC/DC Example: A XOR B XOR C

Test case number	*A*	B	C	*decision*
1	F	F	F	F
2	F	F	T	T
3	F	T	F	T
4	F	T	T	F
5	T	F	F	T
6	T	F	T	F
7	T	T	F	F
8	T	T	T	T

Potential tests:
A: {T1, T5}, {T2, T6}, {T3, T7}, {T4, T8}

Insight: change of any condition will affect the decision

Baseline + Toogle Shortcut Method

- Disclaimer: does not always work
  - Masking, shortcut evaluation, …
- Choose a baseline (e.g., where the decision is True)
- Toggle one condition at a time while keeping the others fixed
- Ensure that the decision flips when that condition changes
- If the decision does not flip, adjust the baseline so that the condition is not masked

MC/DC Example: A XOR B XOR C

Test case number	*A*	B	C	*decision*
1	F	F	F	F
2	F	F	T	T
3	F	T	F	T
4	F	T	T	F
5	T	F	F	T
6	T	F	T	F
7	T	T	F	F
8	T	T	T	T

Potential tests:
{T8}

Baseline where the decision is True

MC/DC Example: A XOR B XOR C

Test case number	*A*	B	C	*decision*
1	F	F	F	F
2	F	F	T	T
3	F	T	F	T
4	F	T	T	F
5	T	F	F	T
6	T	F	T	F
7	T	T	F	F
8	T	T	T	T

Flip for A

Potential tests:
{T8, T4}

MC/DC Example: A XOR B XOR C

Test case number	*A*	B	C	*decision*
1	F	F	F	F
2	F	F	T	T
3	F	T	F	T
4	F	T	T	F
5	T	F	F	T
6	T	F	T	F
7	T	T	F	F
8	T	T	T	T

Flip for B

Potential tests:
{T8, T4, T6}

MC/DC Example: A XOR B XOR C

Test case number	*A*	B	C	*decision*
1	F	F	F	F
2	F	F	T	T
3	F	T	F	T
4	F	T	T	F
5	T	F	F	T
6	T	F	T	F
7	T	T	F	F
8	T	T	T	T

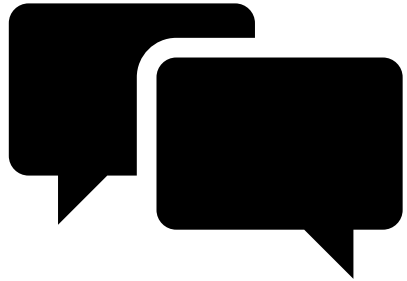
Flip for C

Potential tests:
{T8, T4, T6, T7}

MC/DC Coverage

- Each decision takes every possible outcome
  - Decision: Boolean expression consisting of conditions and zero or more operators
    - In contrast to branches, also covers assignments to Boolean variables
- Each condition in a decision takes every possible outcome
- Each condition in a decision is shown to independently affect the outcome of the decision

Course Exercise



Besides testing, what else can we do to ensure the quality of our implementation?

https://pollev.com/manuelrigger

Quality Assurance

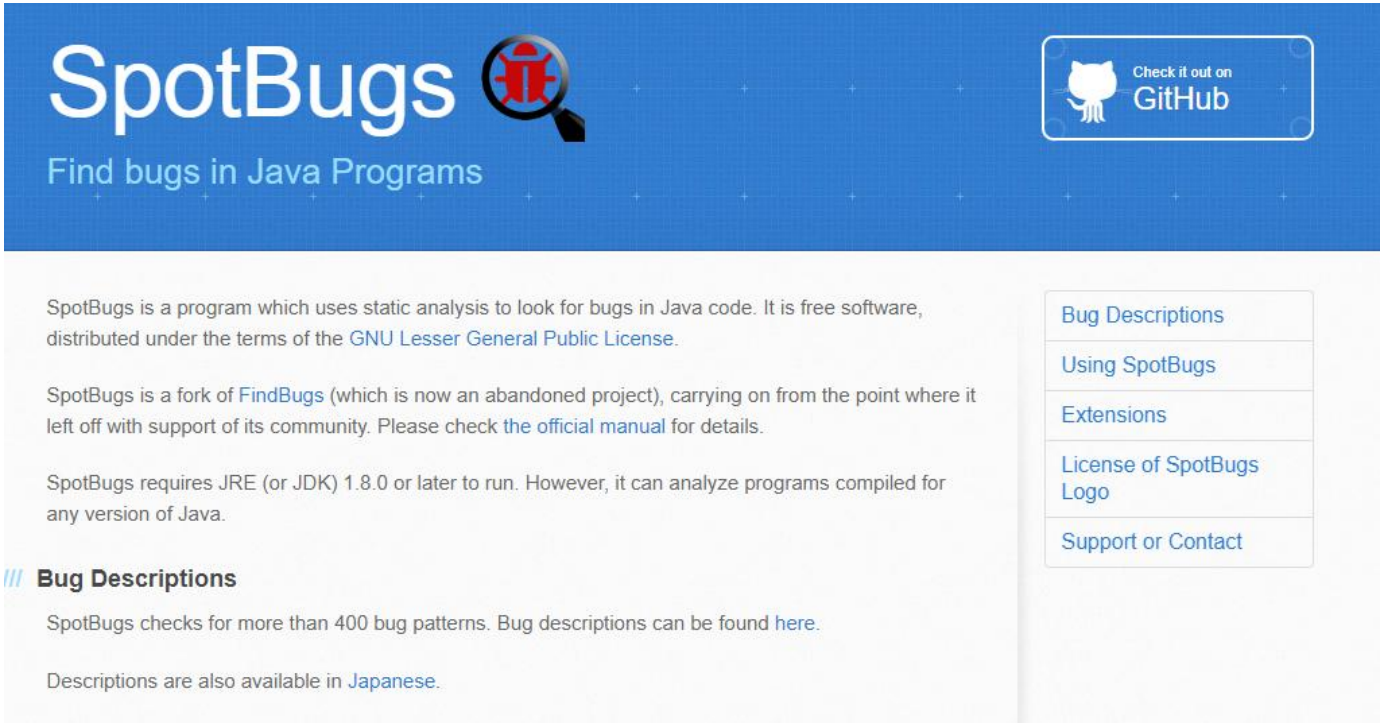
- Peer reviews (e.g., on pull requests, design documents, …)
- Coding standards and best practices
- Continuous integration
- **Static analysis**

Static Analysis

Static Analysis

- Static analysis tools are tools that reason about code *without* executing it
- Linters and style checkers: check the formatting and practices in the code without a deeper analysis
- Finding deeper bugs by reasoning about executions and patterns
  - Data-flow analysis: reason about the potential set of values at different points in the execution
  - Control-flow analysis: reasons about the potential orders of executions

Static Analysis: Examples for Tools



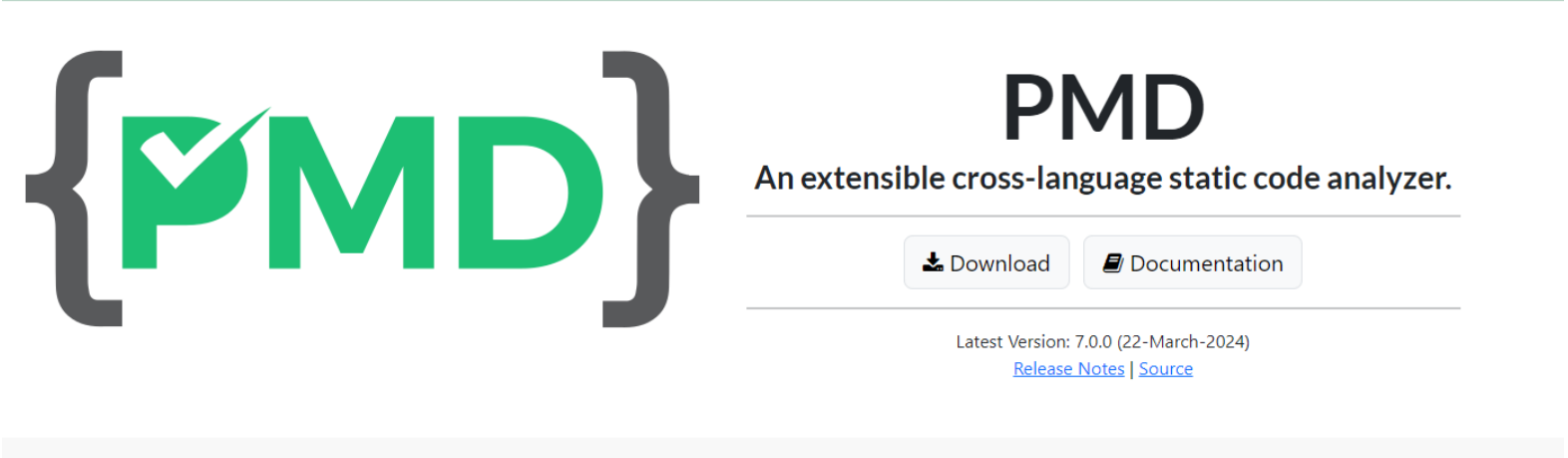
https://spotbugs.github.io/

Static Analysis: Examples for Tools

RV: Random value from 0 to 1 is coerced to the integer 0 (RV_01_TO_INT)

A random value from 0 to 1 is being coerced to the integer value 0. You probably want to multiply the random value by something else before coercing it to an integer, or use the `Random.nextInt(n)` method.

Static Analysis: Examples for Tools



https://docs.pmd-code.org/latest/

Static Analysis: Examples for Tools

AvoidDeeplyNestedIfStmts

Since: PMD 5.5.0

Priority: Medium (3)

Avoid creating deeply nested if-then statements since they are harder to read and error-prone to maintain.

This rule is defined by the following Java class: net.sourceforge.pmd.lang.apex.rule.design.AvoidDeeplyNestedIfStmtsRule

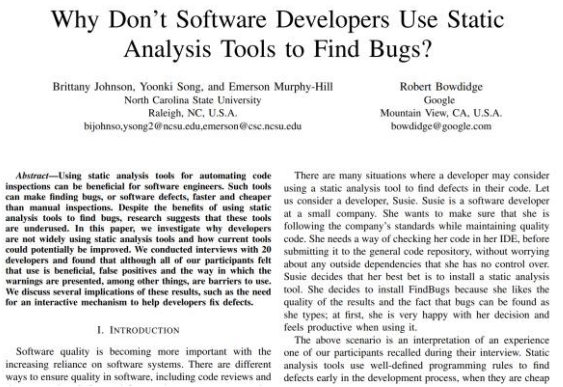
Example(s):

```
public class Foo {
    public void bar(Integer x, Integer y, Integer z) {
        if (x>y) {
            if (y>z) {
                if (z==x) {
                    // !! too deep
                }
            }
        }
    }
}
```

https://pmd.sourceforge.io/pmd-6.0.0/pmd_rules_apex_design.html#avoiddeeplynestedifstmts

Bad Reputation: Reasons

- Can be slow
- Can result in false positives (i.e., false alarms)
- Can result in false negatives (i.e., missed bugs)
- Warnings might not be sufficiently actionable (i.e., developers do not know how to address them)
- Can be difficult to integrate into the developer's workflow
- Might be inapplicable for complex features (e.g., when they are not modeled by the tool)



Why Don't Software Developers Use Static Analysis Tools to Find Bugs?

Brittany Johnson, Yoonki Song, and Emerson Murphy-Hill
North Carolina State University
Raleigh, NC, U.S.A.
bijohnso,ysong2@ncsu.edu,emerson@csc.ncsu.edu

Robert Bowdidge
Google
Mountain View, CA, U.S.A.
bowdidge@google.com

***Abstract*—Using static analysis tools for automating code inspections can be beneficial for software engineers. Such tools can make finding bugs, or software defects, faster and cheaper than manual inspections. Despite the benefits of using static analysis tools to find bugs, research suggests that these tools are underused. In this paper, we investigate why developers are not widely using static analysis tools and how current tools could potentially be improved. We conducted interviews with 20 developers and found that although all of our participants felt that use is beneficial, false positives and the way in which the warnings are presented, among other things, are barriers to use. We discuss several implications of these results, such as the need for an interactive mechanism to help developers fix defects.**

I. Introduction

Software quality is becoming more important with the increasing reliance on software systems. There are different ways to ensure quality in software, including code reviews and

There are many situations where a developer may consider using a static analysis tool to find defects in their code. Let us consider a developer, Susie. Susie is a software developer at a small company. She wants to make sure that she is following the company's standards while maintaining quality code. She needs a way of checking her code in her IDE, before submitting it to the general code repository, without worrying about any outside dependencies that she has no control over. Susie decides that her best bet is to install a static analysis tool. She decides to install FindBugs because she likes the quality of the results and the fact that bugs can be found as she types; at first, she is very happy with her decision and feels productive when using it.

The above scenario is an interpretation of an experience one of our participants recalled during their interview. Static analysis tools use well-defined programming rules to find defects early in the development process, when they are cheap

Static Analysis at Google and Facebook

DOI:10.1145/3338112

Key lessons for designing static analyses tools deployed to find bugs in hundreds of millions of lines of code.

BY DINO DISTEFANO, MANUEL FÄHNDRICH, FRANCESCO LOGOZZO, AND PETER W. O'HEARN

Scaling Static Analyses at Facebook

STATIC ANALYSIS TOOLS are programs that examine, and attempt to draw conclusions about, the source of other programs without running them. At Facebook, we have been investing in advanced static analysis tools that employ reasoning techniques similar to those from program verification. The tools we describe in this article (Infer and Zoncolan) target issues related to crashes and to the security of our services, they

integrated in the workflow used by security engineers. It has led to thousands of fixes of security and privacy bugs, outperforming any other detection method used at Facebook for such vulnerabilities. We will describe the human and technical challenges encountered and lessons we have learned in developing and deploying these analyses.

There has been a tremendous amount of work on static analysis, both in industry and academia, and we will not attempt to survey that material here. Rather, we present our rationale for, and results from, using techniques similar to ones that might be encountered at the edge of the research literature, not only simple techniques that are much easier to make scale. Our goal is to complement other reports on industrial static analysis and formal methods,[1,6,13,17] and we hope that such perspectives can provide input both to future research and to further industrial use of static analysis.

Next, we discuss the three dimensions that drive our work: bugs that matter, people, and actioned/missed bugs. The remainder of the article describes our experience developing and deploying the analyses, their impact, and the techniques that underpin our tools.

Context for Static Analysis at Facebook

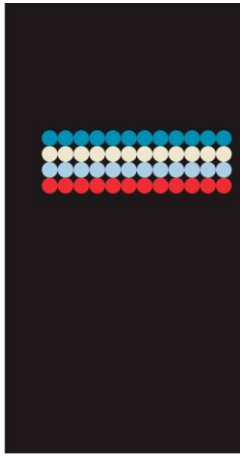
Bugs that Matter. We use static analysis to prevent bugs that would affect our products, and we rely on our engineers' judg-

https://cacm.acm.org/magazines/2019/8/238344-scaling-static-analyses-at-facebook/fulltextx

For a static analysis project to succeed, developers must feel they benefit from and enjoy using it.

BY CAITLIN SADOWSKI, EDWARD AFTANDILIAN, ALEX EAGLE, LIAM MILLER-CUSHON, AND CIERA JASPAN

Lessons from Building Static Analysis Tools at Google



SOFTWARE BUGS COST developers and software companies a great deal of time and money. For example, in 2014, a bug in a widely used SSL implementation ("goto fail") caused it to accept invalid SSL certificates,[36] and a bug related to date formatting caused a large-scale Twitter outage.[23] Such bugs are often statically detectable and are, in fact, obvious upon reading the code or documentation yet still make it into production software.

Previous work has reported on experience applying bug-detection tools to production software.[6,3,7,29] Although there are many such success stories for developers using static analysis tools, there are also reasons engineers do not always use static analysis

Not integrated. The tool is not integrated into the developer's workflow or takes too long to run;

Not actionable. The warnings are not actionable;

Not trustworthy. Users do not trust the results due to, say, false positives;

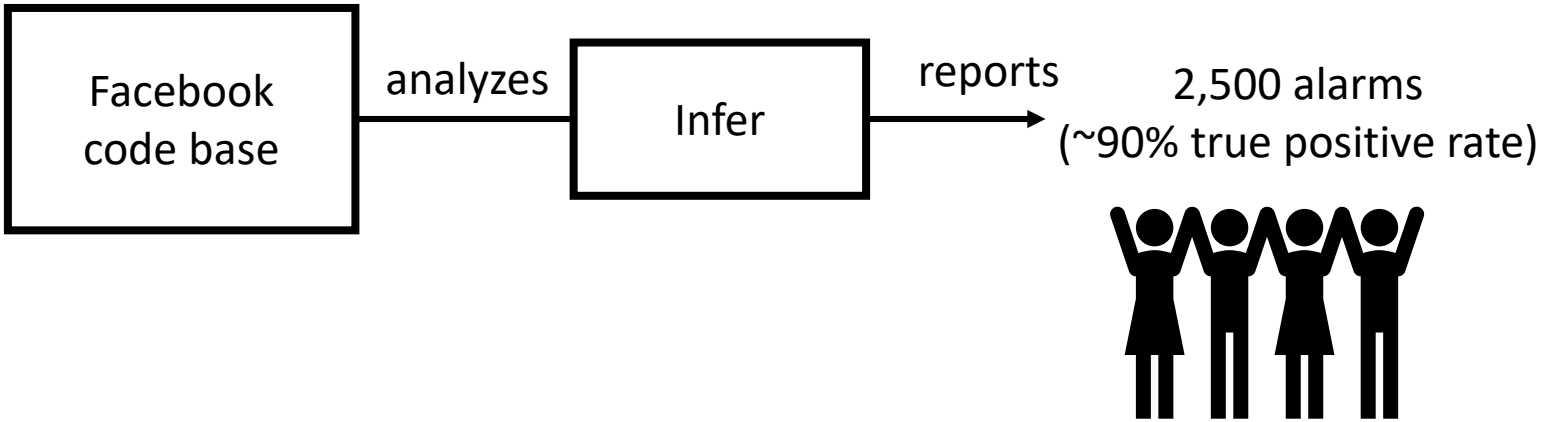
Not manifest in practice. The reported bug is theoretically possible, but the problem does not actually manifest in practice;

» key insights

- **Static analysis authors should focus on the developer and listen to their feedback.**
- **Careful developer workflow integration is key for static analysis tool adoption.**
- **Static analysis tools can scale by**

https://research.google/pubs/lessons-from-building-static-analysis-tools-at-google/

Scenario


Facebook
code base
analyzes
Infer
reports
2,500 alarms
(~90% true positive rate)

Deployment of Infer at Facebook

[…] Infer would be run ***once per night*** *on the entire Facebook Android codebase, and it would generate a list of issues. We* ***manually looked at the issues****, and* ***assigned them to the developers we thought best able to resolve them****. The response was stunning: we were greeted by near silence. We assigned 20–30 issues to developers, and* ***almost none of them were acted on****. We had worked hard to get the* ***false positive rate down to what we thought was less than 20%****, and yet the fix rate—the proportion of reported issues that developers resolved—was near zero.*

Subsequent Deployment of Infer at Facebook

*Next, we switched Infer on at **diff time**. The response of engineers was just as stunning: **the fix rate rocketed to over 70%**. The same program analysis, with same false positive rate, had much greater impact when deployed at diff time.*

Static Analysis on Diffs

- Rather than reporting issues for the whole codebase, only issues for the changed code are reported
- Integrated during code review as bots
- Those who changed the code are prepared to look and discuss changes
  - Timeliness of reports is important!

Initial Deployment of FindBugs at Google

Attempt 1. Bug dashboard. Initially, in 2006, FindBugs was integrated as a centralized tool that ran ***nightly over the entire Google codebase****, producing a database of findings engineers could examine through a dashboard. Although FindBugs found hundreds of bugs in Google's Java codebase, the dashboard* ***saw little use*** *because a bug dashboard was* ***outside the developers' usual workflow****, and* ***distinguishing between new and existing static-analysis issues was distracting****.*

Initial Deployment of FindBugs at Google

Attempt 2. Filing bugs. The BugBot team then began to ***manually triage new issues*** *found by each nightly FindBugs run,* ***filing bug reports for the most important ones****. In May 2009, hundreds of Google engineers participated in a* ***companywide "Fixit" week****, focusing on addressing FindBugs warnings. They reviewed a total of 3,954 such warnings (42% of 9,473 total),* ***but only 16% (640) were actually fixed****, despite the fact that 44% of reviewed issues (1,746) resulted in a bug report being filed. Although the Fixit validated that many issues found by FindBugs were actual bugs,* ***a significant fraction were not important enough to fix in practice****. Manually triaging issues and filing bug reports is not sustainable at a large scale.*

Problems with “Old” bugs

- Context switching might be costly for developers
- Might not be relevant to their current work or project
- It can be difficult to assign the bug to the right person (e.g., might have even left the company)

Subsequent Deployment of FindBugs at Google

Attempt 3. Code review integration. The BugBot team then implemented a system in which FindBugs ***automatically ran when a proposed change was sent for review****, posting results as comments on the code-review thread, something the code-review team was already doing for style/formatting issues. Google developers could suppress false positives and apply FindBugs' confidence in the result to filter comments. The tooling further attempted to show only new FindBugs warnings* ***but sometimes miscategorized issues as new****. Such integration was discontinued when the code-review tool was replaced in 2011 for two main reasons****: the presence of effective false positives caused developers to lose confidence in the tool****, and developer customization resulted in an inconsistent view of analysis results.*

Tricorder

- Tricorder: Google’s current static analysis platform
- Came out of several failed attempts to integrate static analysis with the developer workflow at Google
- Key difference: deliver only valuable results to its users

Successful Static Analysis

- Google: “*Focus on developer happiness*”
- Must be understandable: users can understand the output
- Must be actionable and easy to fix: the report should guide the user to fix it (or automatically fix)
- Should have a low false positive rate (less than 10%)
- Have the potential for significant impact on code quality
- Should integrate with the developers’ workflow
- Reports should be timely

Conclusion on Static Analysis

- Static analysis can be useful, but it can be more challenging to integrate it to bring value